On Combiners for Verifiable Random Functions

Basil Rohner (brohner@ethz.ch)
Bachelor's Thesis, ETH Zürich
April 11, 2026

Research Question

Can we *combine* multiple instances of verifiable random function (VRFs) into a single VRF that is secure as long as *enough* of the instances are?

- How do we *combine* the candidate schemes?
- What are *enough* candidates to combine?

Can we combine *k* out of *n* VRF instances in a *black-box* fashion?

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

A VF is a purely syntactic object, we define security properties on top of it.

Properties of Verifiable Functions

ε-Correctness

Honestly generated image-proof pairs verify with probability at least ε:

$$\Pr\left[\begin{array}{l}(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda}) \\ \quad x \leftarrow_{\$} \{0,1\}^{\lambda} \quad : \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk},x,y,\pi) = 1 \\ (y,\pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk},x)\end{array}\right] \geq \varepsilon(\lambda)$$

VF is *correct* if it is 1-correct and *statistically correct* if it is $(1-\varepsilon)$-correct and $\varepsilon \in \mathsf{negl}(\lambda)$.

Properties of Verifiable Functions

Unique Provability

For every pk and every x, at most one image y is provable:

$$\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_0, \pi_0) = \mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1.$$

Properties of Verifiable Functions

$(\varepsilon, \mathfrak{t})$-Pseudorandom

For any adversary $\mathcal{A}$ running in time at most $\mathfrak{t}$, playing the prf experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$: The advantage $\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ is bounded from above by

$$\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Verifiable Random Functions — why

- VRFs are a **public-key analogue of PRFs** with a verifiability guarantee.
- Key applications:
  - distributed leader election in Proof-of-Stake blockchains,
  - lottery and e-cash systems,
  - resettable zero-knowledge proofs,
  - signatures with strong verifiability.
- Weaker variants: *weak VRFs* (random challenge) and *verifiable unpredictable functions* (image hard to predict, not pseudorandom).

Robust Black-Box Combiners

Motivation. Constructions rely on hardness assumptions that may break. We want to *hedge* our bets by combining several candidate schemes so security survives as long as enough candidates are secure.

Definition (Robust $(n; k)$ Black-Box P-Combiner)

A ppt. oracle Turing machine Π_{P} that, given oracle access to n candidates $\mathcal{O}_1, \ldots, \mathcal{O}_n$, implements primitive P securely whenever at least k of the candidates do.

- **Black-box:** the combiner queries candidates as oracles — it cannot inspect or modify their code.
- We require security against adversaries with access to a **PSPACE oracle**, forcing the combiner to derive its security from the candidates.
- Known results: simple $(n; 1)$ black-box combiners exist for PRFs, OWFs, PKE, key agreement, … — but **not known for VRFs**.

Robust Black-Box DAG Combiners

Idea. Represent each algorithm of the combiner as a *directed acyclic graph* (DAG) $\mathcal{G}$ that makes the structure of oracle calls explicit.

- Dedicated **source** s (the input) and **sink** t (the output).
- Every other vertex v carries
  - a label $\ell(v) = (i, j)$ specifying *which candidate's* j-th oracle is queried, and
  - a ppt. Turing machine f^v that computes the query input from the responses of the predecessors $\mathcal{N}_{\mathcal{G}}^{-}(v)$.
- Execution traces the DAG in topological order: at each v, build $\mathbf{q}_v \leftarrow f^v(\{\mathbf{x}_w\}_{w \in \mathcal{N}_{\mathcal{G}}^{-}(v)})$ and set $\mathbf{x}_v \leftarrow \mathcal{O}_{\ell(v)}(\mathbf{q}_v)$. The sink's Turing machine f^t produces the final output.

Equivalence. Every robust $(n; k)$ black-box combiner has a *computationally equivalent* DAG combiner — the DAG form is a convenient normal form for our proofs.

Equivalence

- Robust $(n; k)$ black-box DAG combiners are robust $(n; k)$ black-box combiners
- Robust $(n; k)$ black-box combiners can be converted into robust $(n; k)$ black-box DAG combiner

By induction over the number of oracle queries, bounded from above by $p(\lambda) \in \text{poly}$. $p(\lambda)$ layers. At every vertex v in layer l simulate the combiner Π_{VRF}

Impossibility of Robust (2; 1) Black-Box DAG VRF-Combiners

Impossibility of Robust (2; 1) Black-Box DAG VRF-Combiners

Impossibility of Robust (2; 1) Black-Box DAG VRF-Combiners

Impossibility of Robust $(n; \lfloor n/2 \rfloor)$ Black-Box DAG VRF-Combiners

Impossibility of Robust $(n; n-1)$ Black-Box DAG VRF-Combiners

What We Showed

DAG combiners are a normal form

Every robust $(n; k)$ black-box combiner has a computationally equivalent robust $(n; k)$ black-box DAG combiner — we can reason over DAGs without loss.

Impossibility

There is **no** robust $(2; 1)$ black-box VRF-combiner.
By reduction, no robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner exists either — at least half of the candidates must be secure.

Matching positive result

If we *strengthen* the candidates by assuming every one is uniquely provable and perfectly correct, then a simple XOR-style robust $(n; 1)$ black-box VRF-combiner exists.

Takeaway. Combining VRFs is strictly harder than combining PRFs: verifiability

Proof Sketch (1/3)

placeholder for proof

Proof Sketch (2/3)

placeholder for proof

Proof Sketch (3/3)

placeholder for proof

Beyond Impossibility

The impossibility result rules out black-box VRF-combiners in the strictest setting. Several natural relaxations are worth exploring:

- **Statistical correctness.** Require the combined scheme to be correct only with overwhelming probability, rather than perfectly — does this open up new constructions?
- **Stronger candidates.** Our positive result assumes *all* candidates are uniquely provable and perfectly correct. How far can we weaken this (e.g. statistical uniqueness, a mix of assumptions) and still combine?
- **Richer VRF notions.** Combiners for VRF variants with additional algebraic structure:
  - constrainable VRFs,
  - key-homomorphic VRFs,
  - aggregable VRFs,

Open Questions

Conjecture: $(n; n-1)$ is also impossible

There exists no robust $(n; n-1)$ black-box VRF-combiner.

That is, it is *not* enough for all-but-one candidates to be secure — any robust black-box VRF-combiner needs strictly more than a constant number of insecure candidates to tolerate.

Why it is hard. Our $(2; 1)$ impossibility proof exploits a specific structural argument about obscure queries and verification traces. Extending it to $n-1$ adversarial candidates would require reasoning about the simultaneous freedom on many oracles, and the current technique does not directly generalize.

Other open directions.

- Closing the gap between $(n; \lfloor n/2 \rfloor)$ (impossible) and $(n; n-1)$ (conjectured impossible).
- Finding the weakest candidate assumptions under which *some* robust black-box